

Lecture 15

Elm Basics

CS 12 Team 25.2

A.Y. 2025–2026, 2nd Sem

Department of Computer Science

College of Engineering

University of the Philippines Diliman

Sanity Check

- Attendance?
- Audible at the back?
- TV(s) on?
 - (we're working on getting the other TV fixed...)
- **Recording?**
- Important announcements?

The Elm Programming Language



- <https://elm-lang.org/> (Evan Czaplicki, 2012)
- Functional programming language for front-end web apps
- Purely immutable, statically typed
- Safe type system; practically no runtime errors

Hello World in Elm

- On the browser (<https://ellie-app.com/>):
- Program entrypoint is `main` definition
- `import Html` allows the `Html` module to be used
- `Html.text` is a function inside the `Html` module
 - Produces web page text

Built-in Data Types

<i>Type</i>	<i>Values</i>
Bool	True, False
Int	-140, 0, 150
Float	1.25, -1.0

Built-in Data Types

<i>Type</i>	<i>Values</i>
-------------	---------------

Char	'a', '\', \u{1F648}, '🙈'
------	--------------------------

String	"hello", "a\nb\nc", "", "\",
--------	------------------------------

```
"""multi  
line  
string"""
```

Common Operators

<i>Purpose</i>	<i>Operators</i>	<i>Notes</i>
Arithmetic	<code>+</code> <code>-</code> <code>*</code> <code>/</code> <code>//</code> <code>^</code>	Operands: <code>Int</code> or <code>Float</code> <code>/</code> returns <code>Float</code> <code>//</code> returns <code>Int</code> <code>2 ^ 3 ^ 2 == 512</code> <code>(2 ^ 3) ^ 2 == 64</code>

<i>Purpose</i>	<i>Operators</i>	<i>Notes</i>
Comparison	<code>==</code> <code>/=</code> <code><</code> <code><=</code> <code>></code> <code>>=</code>	Unconventional inequality operator <code>/=</code>
Logical	<code>&&</code> <code> </code>	<code>not</code> is a function, operands are <code>Bool</code>
Grouping	<code>()</code>	Link to operator precedence

Calling functions

- Simply place arguments after function name (no parentheses)
- Examples:
 - `toFloat 1` returns `Float` value `1`
 - `round 1.5` returns `Int` value `2`
 - `String.fromInt 1` returns `String` value `"1"`
 - `String.fromFloat 0.1` returns `String` value `"0.1"`
 - `Debug.toString True` returns `String` value `"True"`
 - `min 1.2 1.55` returns `Float` value `1.2`

Calling functions

- More built-in functions: <https://package.elm-lang.org/packages/elm/core/latest/Basics>

Global variable declaration

- Variables must start with a lowercase letter (ideally camelCase):
- Must not be indented:

```
-- Invalid  
    isWeekend = False
```

“Variables” and Reassignment

Important: Variables cannot be reassigned to

- Elm variables do not vary:

```
year = 2023  
year = year + 1  -- Invalid in Elm!
```

Since Elm has no reassignment, what does that disallow us from doing?

“Variables” and Reassignment

Important: Elm does not have while/for loops

- Python-style loops cannot exist in Elm

```
while k < n:
```

```
    ...
```

```
    k += 1
```

```
for i in range(n): # Elm has no built-in for loop like this
```

```
    ...
```

“Variables” and Reassignment

Important: Elm does not have while/for loops

- You will be forced to think of solutions that do not make explicit use of loops or repetition

Statements vs Expressions

Statement: Does not evaluate to a value; cannot be assigned to a variable

- if-else statement in Python

```
if isWeekend:  # Cannot be assigned to a variable
    mood = "happy"
else:
    mood = "sad"
```

Statements vs Expressions

Expression: Evaluates to a value; can be assigned to a variable

- Ternary conditional expression in Python

```
"happy" if isWeekend else "sad"
```


Statements vs Expressions

Note: Almost everything is an expression in Elm

- Variable declaration is a statement

Conditionals: If Expression

- `if` is an expression in Elm

```
mood =  
    if isWeekend then  
        "happy"           -- Branches must also  
    else                  -- be expressions  
        "sad"
```

- Can be a one-liner

```
mood = if isWeekend then "happy" else "sad"
```

Conditionals: If Expression

- Has `else` if chaining

```
mood =  
    if isRaining then  
        "gloomy"  
    else if hasExam then  
        "scared"  
    else if isWeekend then  
        "happy"  
    else  
        "sad"
```

Conditionals: Case Expression

- case expression performs pattern matching on given:

```
day =  
  case n of  
    1 -> "Sunday"           -- Patterns must be more indented  
    2 -> "Monday"          -- than `case` line  
    3 -> "Tuesday"  
    4 -> "Wednesday"       -- All patterns must have same  
    5 -> "Thursday"        -- level of indentation  
    6 -> "Friday"  
    7 -> "Saturday"  
    _ -> "Unknown day"     -- `_` is catch-all pattern
```

Conditionals: Case Expression

```
day =  
    if n == 1 then "Sunday"  
    else if n == 2 then "Sunday"  
    else if n == 3 then "Monday"  
    else if n == 4 then "Wednesday"  
    else if n == 5 then "Thursday"  
    else if n == 6 then "Friday"  
    else if n == 7 then "Saturday"  
    else "Unknown day"
```

- Very useful; more on this later

Let Expression and Scoping

- let expression syntax

```
let
  varName1 = expression1  -- Must be more indented
  varName2 = expression2  -- than `let`
  -- ...
  varNameN = expressionN
in
bodyExpression            -- Single expression
```

Let Expression and Scoping

- Defined variables are not visible outside `let`

```
z =  
  let  
    x = 11  
    y = x + 1  -- Can refer to previous declarations  
  in  
    ----- Can start using `x` and `y` starting here...  
    x + y  
    ----- ...up to here (only within body expression)  
  
-- Cannot use `x` and `y` here (out of scope)
```

Shadowing

- Shadowing will be flagged

```
mood = "excited"                                -- Globally defined `mood`

yourMood =
  let
    mood =
      if isRaining then
        "gloomy"
      else if hasExam then
        "scared"
      else if isWeekend then
```

-- This `mood` definition
-- shadows the `mood`
-- definition in outer
-- scope; flagged


```
        "happy"  
    else  
        "sad"  
in  
mood
```

Shadowing

- Shadowing is usually a cause of subtle bugs

Declaring Functions

- Function declaration syntax

```
functionName arg1 arg2 ... argN =  
    functionBody                -- Single expression
```

- Unary function example

```
double n =  
    n * 2
```

Declaring Functions

- Binary function example

```
combination n k =  
  let  
    num = factorial n  
    den = (factorial k) * (factorial (n - k))  
  in  
    num // den
```

Type Checking and Type Inference

- Previous example

```
-- Assume that `factorial` takes in an `Int`  
-- and returns an `Int`  
  
combination n k =  
    let  
        num = factorial n  
        den = (factorial k) * (factorial (n - k))  
    in  
    num // den
```

- Elm can infer the following:
 - `combination` returns `Int` (implied by `//`)
 - `n` is an `Int` (implied by `factorial n`)
 - `k` is an `Int` (implied by `factorial k`)
- Elm can detect the type error here:

```
-- Second argument must be `Int`, not `String`;  
-- will not compile  
combination 4 "2"
```

Function Signatures

- Function signature syntax:

`functionName: TypeArg1 -> TypeArg2 -> ... -> ReturnType`

Function Signatures

- Previous example with signatures:

```
double : Int -> Int
double n =
  n * 2

combination : Int -> Int -> Int
combination n k =
  let
    num = factorial n
    den = (factorial k) * (factorial (n - k))
  in
    num // den
```


Pipeline Operator

$x \mid \!> f$: same as $f \ x$ (visualize as direction of data flow)

- Useful for expressing program as sequence of transformations

Pipeline Operator

```
usual =  
    List.reverse (List.range 1 5 lst)  
  
pipelined =  
    lst  
    |> List.range 1 5      -- `lst` is passed as last arg  
    |> List.reverse       -- Result of above is passed as last arg
```

- Makes flow very readable; commonly used

Pipeline Operator

`f <|` x: Same as `f x` (also visualize direction of data flow)

```
withParens =  
  List.reverse (List.range 1 5 lst)  
  
withoutParens =  
  List.reverse <| List.range 1 5 lst
```

- Not as useful as `|>`